

Pandas & NumPy Practice Problem Set

Comprehensive Data Analysis & Manipulation Exercises across 6 Real-World Datasets

Overview: This structured problem set covers the complete Pandas & NumPy pipeline—from basic filtering and missing value handling to complex string parsing, custom aggregations, rolling windows, and multi-table operations. Solve these questions without looking at solutions to validate your data analysis skills!

1. bengaluru_house_prices.csv

Focus: Cleaning, String Parsing, Outliers, Transform

EASY

1. Filter all rows where `location` is `'Electronic City Phase II'` and `price` is greater than `40`.
2. Find the total count and percentage of missing (`NaN`) values for each column in the dataset.

MEDIUM

1. The `size` column contains string representations like `'2 BHK'` and `'4 Bedroom'`. Extract the integer numeric value from this column into a new column called `bhk`.
2. Fill missing values in the `balcony` column using the median balcony count grouped by `bhk`.

ADVANCED

1. The `total_sqft` column contains single values, range values (e.g., `'2100 - 2850'`), and invalid string characters. Write a custom function with `.apply()` to convert ranges to their average float value, convert standard numerical strings to floats, and turn invalid entries into `NaN`.
2. Calculate `price_per_sqft` using $(price \times 100,000) / total_sqft$. Then, use `.groupby()` and `.transform()` to filter out rows where `price_per_sqft` falls outside 1 standard deviation ($\mu \pm 1\sigma$) of its respective `location` group.

2. diabetes.csv

Focus: Domain Invalidation, Quantile Binning, Conditional Logic

EASY

1. Compute summary statistics (`describe()`) separately for diabetic (`Outcome == 1`) and non-diabetic (`Outcome == 0`) patients.
2. Count how many records have a `Glucose` level of `0` and a `BMI` of `0`.

MEDIUM

1. In clinical data, `0` values in `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI` represent missing measurements. Replace all `0`s in these specific columns with `np.nan`.
2. Bin `Age` into discrete groups using `pd.cut()` with intervals: `['Under 30', '30-50', '50+']`. Fill the missing `Insulin` values with the median `Insulin` value computed per `Age_Group` and `Outcome`.

ADVANCED

1. Create a categorical column `Risk_Level` using `np.select()` based on complex conditions involving thresholds of `BMI`, `Glucose`, and `Age`.
2. Group the dataset by `Age_Group` and calculate the relative percentile rank of each patient's `BMI` within their respective age group using `.groupby()['BMI'].rank(pct=True)`.

3. ipl-matches.csv

Focus: Datetime, Crosstabs, Nested Lists, Cumulative Aggs

EASY

1. Convert `Date` to a `datetime64` column and filter all matches played in the year `2022`.
2. Find the top 5 venues that hosted the highest total number of IPL matches.

MEDIUM

1. Construct a cross-tabulation matrix using `pd.crosstab()` between `TossWinner` and `WinningTeam`. Calculate the percentage of matches where the toss winner won the match.
2. Filter matches from seasons post-2018 and find which player won the most `Player_of_Match` awards in each individual season.

ADVANCED

1. The `Team1Players` and `Team2Players` columns contain string representations of Python lists. Safely evaluate and flatten these strings across all matches to find the top 10 most frequently featured players in IPL history.
2. Sort the dataset by `Date` and calculate the cumulative rolling win total for each team across all seasons using `.groupby()` and `.cumsum()`.

4. movies.csv

Focus: String Splitting, Exploding, Regex, Pivot Tables

EASY

1. Sort movies by `imdb_rating` in descending order and filter for movies released after 2010 with a `runtime` over 120 minutes.
2. Identify and display all rows where `wins_nominations` is missing (`NaN`).

MEDIUM

1. The `genres` column contains pipe-separated genres (e.g., `'Action|Drama|War'`). Split this column using `.str.split()` and use `.explode()` to calculate the mean `imdb_rating` and total `imdb_votes` for each distinct genre.
2. Extract the exact number of wins from the `wins_nominations` string (e.g., extract 4 from `'4 wins'`) into a new numeric column `wins` using regex with `.str.extract()`.

ADVANCED

1. Construct a `pivot_table` with `year_of_release` as index, top primary genres as columns, and median `imdb_rating` as values. Fill missing cells with 0.
2. Compute the pairwise correlation matrix between `runtime`, `imdb_rating`, `imdb_votes`, and your extracted `wins` count using `.corr()`.

5. Titanic-Dataset.csv

Focus: Feature Extraction, Data Merging, Groupwise Metrics

EASY

1. Calculate the overall survival rate, as well as the survival rate segmented by `Sex` and `Pclass`.
2. Find the count of unique passenger titles extracted from the `Name` column.

MEDIUM

1. Extract social honorifics (e.g., `'Mr'`, `'Mrs'`, `'Miss'`, `'Master'`, `'Dr'`) from `Name` into a new `Title` column using regex.
2. Fill missing `Age` values using the median age calculated specifically for each passenger's extracted `Title`.

ADVANCED

1. Split the DataFrame into two separate DataFrames: `df_demographics` (`PassengerId`, `Name`, `Sex`, `Age`) and `df_ticket` (`PassengerId`, `Pclass`, `Ticket`, `Fare`, `Cabin`, `Embarked`). Recombine them using `pd.merge()` with a `left` join on `PassengerId`.
2. Create a `FamilySize` feature (`SibSp + Parch + 1`). Map this into categories (`'Alone'`, `'Small'`, `'Large'`) and use `.groupby().transform()` to add a `Group_Survival_Rate` column representing the survival rate of each passenger's family group.

6. WineQT.csv

Focus: Concatenation, Quantile Bins, Z-Score Scaling

EASY

1. Filter all wines where `quality >= 7` and `alcohol > 11`.
2. Compute the min, max, mean, and standard deviation of `pH` across each distinct `quality` rating.

MEDIUM

1. Split the dataset into two subsets: `high_quality` (`quality >= 6`) and `low_quality` (`quality < 6`). Add a string column `quality_label` to each, then re-combine them into a single DataFrame using `pd.concat()` along `axis=0`.
2. Bin the `sulphates` column into 3 equal-frequency quantiles using `pd.qcut()` with labels `['Low', 'Medium', 'High']`.

ADVANCED

1. Perform group-wise Z-score standard scaling ($Z = (x - \mu) / \sigma$) on all chemical features (from `fixed acidity` to `alcohol`) grouped by `quality` using `.groupby().transform()`.
2. Identify all row indices that contain at least one chemical feature with an absolute group Z-score exceeding 3 ($|Z| > 3$).